The Field Guide

Market making, backtests and this repo, before the Qminers Algo Trade Arena

2026-11-04

Table of contents

Preface: how to read this **4**

What happens in a round? **5**
The story . . . 5
The idea . . . 6
What this rehearsal teaches . . . 6

What is an exchange? **8**
The story . . . 8
The idea . . . 9
The order book . . . 9
Orders and fills . . . 9
Position, limits and profit . . . 10
What the data looks like . . . 10

What is the price? **12**
The story . . . 12
The idea . . . 12
The mid is noisy . . . 12
The walls know more . . . 13
Robust beats clever . . . 13
A fair value has a timescale . . . 13

Why does a market maker earn anything? **16**
The story . . . 16
The idea . . . 17
The spread is a fee for immediacy . . . 17
What can go wrong: the price moves after you trade . . . 17
So where does the income come from? . . . 18

Who is on the other side? **20**
The story . . . 20
The idea . . . 21
The fills you get are the ones you didn't want . . . 21
Two kinds of market . . . 21
When someone is informed, follow them . . . 21

Why is my backtest lying? **23**
The story . . . 23
The idea . . . 23
Why the error isn't noise . . . 24
The other things a replay assumes . . . 25
What to do about it . . . 25

How do I know a result is real? **27**
The story . . . 27
The idea . . . 27
A grid is a hyperparameter search, with very few folds . . . 27
Hold out days, not ticks . . . 28
Statistics for "the best of many" . . . 28
One grid, read in full . . . 29
When the profit is on one day . . . 29

What does the portal tell me? **31**
The story . . . 31
The idea . . . 31
The twelve figures . . . 31
Calibrate: fitting the simulator to the real thing . . . 32

How is the code laid out? **35**
The story . . . 35
The idea . . . 35
The engine knows no format . . . 35
The bot in three parts . . . 36
The rest of the repo . . . 37

What do I do on the day? **38**
The story . . . 38
The idea . . . 38
The weekend . . . 38
Why the round runs as it does . . . 38
Each round, choose again . . . 39
Risk, once there is a score to protect . . . 40

Appendices **41**

Glossary **41**

Preface: how to read this

This guide is for the Qminers Algo Trade Arena on 7 and 8 November 2026: four five-hour rounds, a Python bot submitted at the end of each. It assumes what an engineer with machine learning behind them already knows (probability, overfitting, hyperparameter search, simulators) and builds the rest from zero: what an order book is, why a market maker earns anything, why a backtest flatters it, and where each of these lives in the repo.

Every chapter answers one question and opens on something that happened: in a Prosperity competition, in a past Qminers hackathon, in one of our own rehearsals or in the literature. Then comes the idea in plain words, at most one or two formulas with a sentence that says each one aloud, and four boxes:

- **If you know ML:** the analogy, and where it breaks.
- **In the repo:** the file, the parameter, the command, the `IDEAS.md` row.
- **On the day:** where `RUNBOOK.md` uses it. The guide gives no instruction RUNBOOK doesn't.
- **Go deeper:** a section or two of the literature, with pages.

Four markets recur, and they are kept apart. *Prosperity* is IMC's competition, whose data the prep used as a stand-in; its prices are in its own credits. *`arena.sim`* is our simulator. *ABIDES* is an independent simulator with informed traders; its prices are in cents. *The mock rounds* are disguised rehearsals with their own portals; the Varde Exchange scores in euros. Nothing measured on one is stated as a fact about the Arena: the Arena's own data comes on the day.

Every number can be checked. A figure comes from `scripts/make_data.py`, which runs a committed command and writes the CSV in `data/` that the figure is drawn from; a number in the text names its `IDEAS.md` row or its source. A figure from a team's own write-up that our engine did not reproduce is marked *their claim*.

The audio edition reads the same text. Each formula is read as its sentence, a table as its summary, and a figure as a short description of it.

What happens in a round?

This chapter is a tour. It uses every term the rest of the guide defines, loosely, so you have the map before the detail. Don't stop on a word you don't know: its chapter is named next to it.

The story

On 3 November we ran a rehearsal of round 1 against an exchange we had never seen, the Varde Exchange. It was built to be unlike anything the repo knew: its own data layout, its own way of calling the bot, prices in cents on a five-cent step, results in euros, and a scoring program written separately from our engine. A fresh agent session got the same prompt it will get on the day and a one-hour clock. Here is what it did (`python/experiments/mock_round/round4/results/DRYRUN_LOG.md`).

- **0:00.** It read the rules: three products, LUMEN, QUILL and SORREL, each with a *position limit* (40, 60 and 30 lots: how much of it you may hold, long or short). Orders *rest* until filled or cancelled. A fee is charged per lot traded *and* per order placed. Three public sessions of data; the score comes from two hidden ones.
- **0:03.** It wrote a *format plug-in*, the piece of code that reads this exchange's files into the engine's own shape (chapter 9), and ran `arena.diagnose`, which describes each product from the data alone: does its price wander or come back to a level, how wide is its *spread* (chapter 2), how often does it jump (chapter 3).
- **0:05.** It wrote the bot: the repo's *template*, with settings for each product, and the *adapter* that speaks this exchange's language (chapter 9). Its first *backtest*, a replay of the public sessions with the bot trading in them, made 730 euros (the prices were in cents: 73,038).
- **0:06.** A *grid*: every combination of a few settings, each backtested, under two *fill models* (chapter 6). One product, SORREL, lost 567 euros quoting; set to take only, it never traded, and dropping that loss took the backtest to about 1,330 euros. `arena.validate` checked the file against the rules (size, speed, no outside libraries).
- **0:07.** It was uploaded. The portal scored it **€367.61** on the public sessions and returned twelve figures and a file of every fill.
- **0:09.** It ran *calibrate*, which searches our engine's settings for one that reproduces the portal's twelve figures (chapter 8). Calibrate **refused**: no setting came within 5%, the closest was 98% off. The session did what the refusal says. It didn't paste a setting into the engine; it rebuilt the score by hand from the fills file. Two things stood out: the fee per order placed had taken 28% of what the bot made before fees, and the portal had filled far fewer lots than any of our settings that traded at all (8,466 against 11,345 to 18,772). The exchange's actual rule, that only half of each trade reached resting orders, was in the rehearsal's answer key; the session didn't pin it down in the time, but it knew not to trust its engine until it did.
- **0:14.** It stopped, as told, after one validated upload and an honest reading of it.

On the hidden sessions the bot scored **€208.77**, against **€80.10** for the example bot the exchange handed out (`python/experiments/mock_round/round4/README.md`).

The idea

A round is a loop (Figure 1), run as many times as five hours and the submission limit allow.

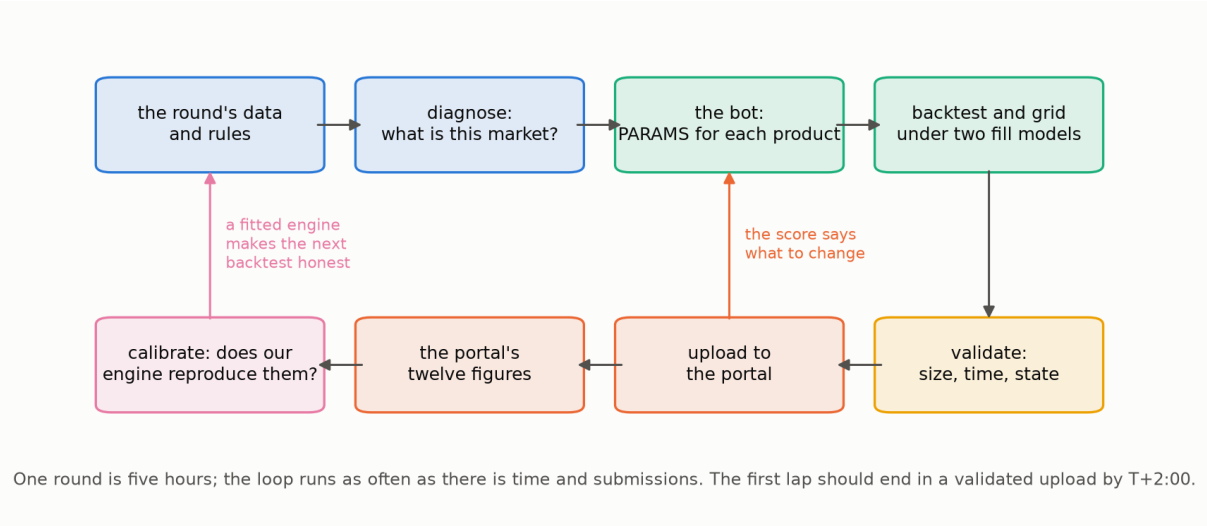


Figure 1: The round's loop. The first lap should end in a validated upload by two hours in (`RUNBOOK.md`, "Each round").

1. **Read the market.** The data is a recording: the *order book* at every *tick* (who is offering to buy and sell, at what prices, how much) and the trades that happened. Chapter 2 is the order book; chapter 3 is the price.
2. **Write the bot.** A bot is called once a tick, sees the book and its own position, and returns orders. Ours is mostly a *market maker*: it offers to buy a little below the fair price and to sell a little above it, and earns the gap, the spread, when others trade with it. Chapter 4 says why that pays at all; chapter 5 says who it loses to.
3. **Backtest.** Replay the recorded days with the bot in them. The replay has to guess what would have happened to orders that weren't really there, and the guess, the fill model, can be worth more than the strategy. Chapter 6.
4. **Choose settings without fooling yourself.** A grid tries many settings; the best of many is flattered by luck, and a round's few days of data are very few. Chapter 7.
5. **Upload, and read what comes back.** The portal is the only judge that matters. Its figures say how far our replay is from the real exchange, and calibrate turns that into engine settings, or refuses. Chapter 8.

Two habits make the loop work. **Upload early**: the first validated upload by two hours in, so there is always something scored. **Change one thing at a time** (the kickoff prompt's rule), and only what the evidence supports, because each upload is one noisy measurement and submissions are limited.

What this rehearsal teaches

The rehearsal's most useful moment was the refusal. A tool that always gives an answer would have handed over the least-bad setting, 98% off, and the next five hours of backtests would have been tuned to an engine that wasn't the exchange's. A tool that says "NO SETTING REPRODUCES THE PORTAL within 5%: don't paste an ENGINE", and says where to look, sent the session to the one place the truth was, the portal's own fills. Most of this guide is about

that distinction: what our tools can measure, what they can only assume, and how to tell the two apart under time pressure.

> **If you know ML**
>
> The loop is model development against a black-box evaluator: a local validation set (the backtest), a hidden test set (the scored sessions), and a limited number of test-set queries (the uploads). The analogy holds well, with two breaks. The local validator is a simulator with a known bias, not a held-out sample of the same data (chapter 6). And every query to the test set costs one of a limited number of submissions, so each is planned like an experiment.

> **In the repo**
>
> - The rehearsal: `python/experiments/mock_round/round4/` (its README rebuilds and scores it in seconds).
> - The loop's tools: `arena.diagnose`, `arena.backtest`, `arena.grid`, `arena.validate`, `experiments.day.calibrate`.
> - The kickoff prompt the session got: `KICKOFF_PROMPT.md`, round 1.

> **On the day**
>
> `RUNBOOK.md`, "Each round", is this loop with times on it. `KICKOFF_PROMPT.md` starts the session that runs it.

> **Go deeper**
>
> - Harris, *Trading and Exchanges*, ch. 13 "Dealers" (from PDF p. 291): what a market maker is, in plain prose, before any of the models.

What is an exchange?

You'll need nothing before this chapter. It defines the order book, orders, fills, the position limit and profit, which every later chapter uses.

The story

The Varde rehearsal's first upload (chapter 1) scored €110.21 on its first public session. The portal showed only totals and a file of fills, and the session rebuilt the rest by hand; the portal's own scoring code, run on the same bot, shows where the number comes from. It is two large numbers that nearly cancel (`data/sessions.csv`, from the rehearsal portal's own scoring):

- **cash: −€2,814.63.** Over the session the bot spent more on buying than it received from selling, fees included.
- **the position, valued: +€2,924.84.** At the end it still held what it had bought and not sold, 17 lots net across its products, and the portal valued those lots at its *settlement price*, the price it chooses for what is still held (the *mark*, below).
- **the score: €110.21,** their sum.

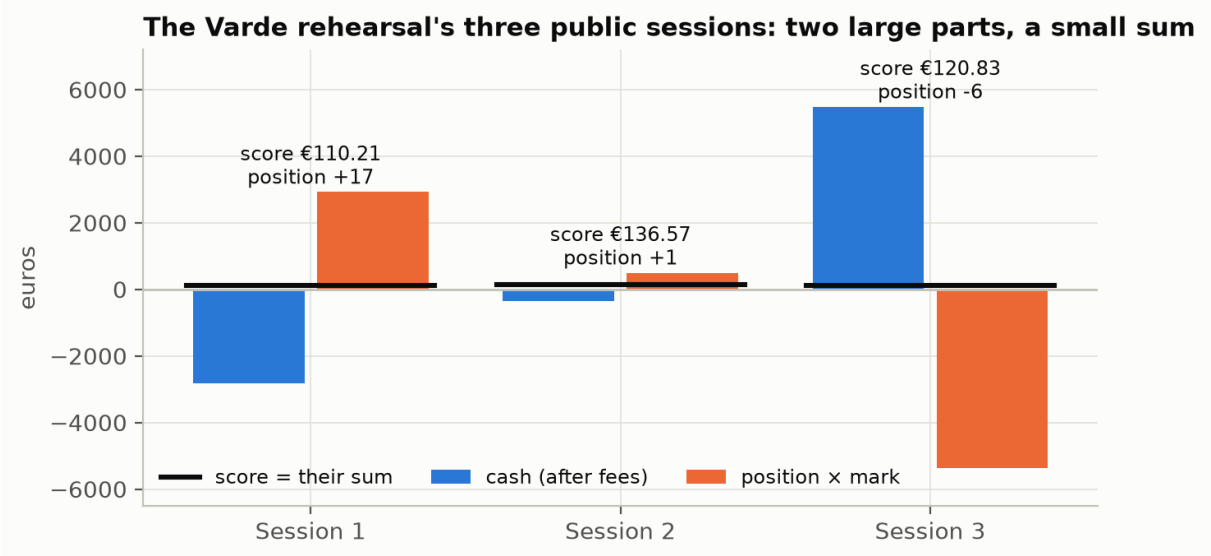


Figure 2: The score's two parts in each of the Varde rehearsal's three public sessions. Data: `data/sessions.csv`, from `python/experiments/mock_round/round4/private/portal.py`'s scoring of the uploaded bot.

Figure 2 shows all three sessions. In sessions 1 and 3 the score is a small difference between two big numbers, in session 2 between two smaller ones, and in every session one of them, the position's value, depends on a price the portal chooses. Hold that thought: it is why chapter 6 counts the *mark* among a backtest's assumptions, and why the end of a session matters more than any single tick.

The idea

The order book

An exchange is a place where offers to trade wait until someone takes them. Each offer is a *limit order*: buy up to this many lots at this price or lower, or sell up to this many at this price or higher. The offers waiting at a moment make up the *order book*: the *bids* (offers to buy) on one side, the *asks* (offers to sell) on the other, each side grouped into *levels*, one per price (Figure 3).

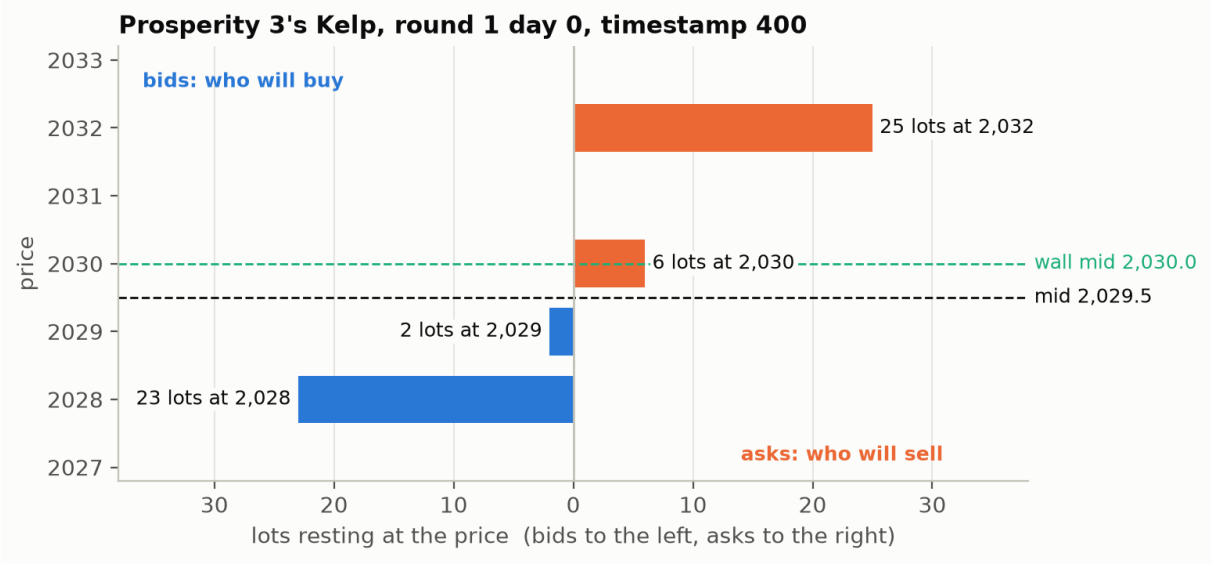


Figure 3: One tick of Prosperity 3's Kelp, round 1, day 0. Data: `data/order_book.csv`, from the engine's loader.

The highest bid is the *best bid* (2,029 here); the lowest ask is the *best ask* (2,030). Together they are the *touch*, and the gap between them is the *spread*, here 1. Prices move in fixed steps, the *tick size*: 1 credit on Prosperity, 5 cents on the Varde Exchange. A spread of one tick is as narrow as a book can be.

The plainest single number for "the price" is the *mid*, halfway between the touch:

$$\text{mid} = \frac{\text{best bid} + \text{best ask}}{2}$$

Here it is 2,029.5. Chapter 3 is about why the mid is often the wrong number to trust, and what the large orders at 2,028 and 2,032, the *walls*, tell you instead.

Orders and fills

A new order either crosses the book or it doesn't.

- A **buy at or above the best ask** (or a sell at or below the best bid) *crosses*. It trades at once with the waiting orders, at their prices, best first. This is *taking*: you pay the spread for trading now.
- An order that **doesn't cross** joins the book and waits. This is *making*, or quoting, and the order is *passive*: you offer liquidity and hope someone takes it. When they do, your order is *filled*, at your price.

Among orders at the same price, the earliest is filled first: *price-time priority*. So a waiting order has a place in a *queue*, and joining a busy level means waiting behind everyone already there.

Exchanges differ in whether a waiting order stays. Prosperity cancels every order at the end of its tick, so a bot re-quotes each tick; the Varde Exchange lets orders *rest* until filled or cancelled. That one rule changes which strategies work (chapter 6).

Position, limits and profit

Your *position* in a product is what you hold: positive after buying more than you sold (*long*), negative after selling more than you bought (*short*; exchanges in these competitions let you sell what you don't hold). Each product has a *position limit*, and exchanges enforce it bluntly. On Prosperity, if a tick's orders for a product could take the position past its limit were they all filled, every one of them is rejected; the Varde Exchange does the same. A bot that quotes the whole room under its limit on both sides can lose all its orders for a tick by miscounting one.

Profit and loss, *PnL*, is what a bot would have if it closed everything now:

$$\text{PnL} = \text{cash} + \text{position} \times \text{price}$$

Cash goes down by the price of everything bought and up by everything sold; fees come out of it. The price that values the position is a choice, which is what the story was about.

What the data looks like

Every exchange records the same market differently, and the repo meets three shapes:

- **Snapshots** (Prosperity): the book at each tick, a few levels a side, plus a file of the trades between ticks, with buyer and seller names when the round reveals them.
- **Prints with an aggressor flag** (the Varde Exchange): each step's book, and each trade marked with which side crossed, so you know whether a buyer or a seller was in a hurry.
- **An event log** (LOBSTER, and many real exchanges): every order added, changed, cancelled or executed, from which any snapshot can be rebuilt.

Whatever the shape, a *format plug-in* reads it into the engine's one internal form, a `Day` of `Ticks`, each with its book and its trades. The engine and every tool work on that, never on the exchange's own files (chapter 9). That is why a new exchange on the day costs a plug-in and an adapter, not a new set of tools.

> **If you know ML**
>
> The order book is the state, a tick is a time step, and the bot is a policy from state to actions. Two things are less familiar. The state is partly made of other agents' intentions, which change when you act; a replay freezes them (chapter 6). And the reward, PnL, is only defined after a choice of price for what you hold, like a metric that needs a reference you have to pick.

> **In the repo**
>
> - The internal form: `python/arena/model.py` (`Book`, `Tick`, `Day`, `Order`, `Fill`).
> - The matching rules: `ARCHITECTURE.md`, "Matching rules (Prosperity semantics)".

- The plug-ins: `python/arena/formats/` (`prosperity.py`, `drill.py`, `snapshots.py`, `orderlog.py`, and the day's `qminers.py`).

! On the day

`ON_THE_DAY.md`, "First hour: answers that change the code", lists what to find out about the Arena's book and orders first: whether orders rest, where a new order joins the queue, how the limit is enforced.

i **Go deeper**

- Harris, *Trading and Exchanges*, ch. 13 "Dealers" (from PDF p. 291).
- Bouchaud et al., *Trades, Quotes and Prices*, ch. 1 (PDF pp. 24–40), which you have read: the ecology of who trades and why.

What is the price?

You'll need the order book, the touch, the mid and the walls (chapter 2), and the market maker and the backtest as chapter 1 introduced them.

The story

In a backtest of Prosperity 4's round 1 in October, our template traded Osmium in one market with three other teams' published bots, all with resting orders. It lost 371 thousand credits. The cause, traced on one day against one of those bots, was one number.

The template prices a product at its *wall mid*, the midpoint of the largest order on each side of the book. At tick 2,400 of day 0 the real wall on the bid side was 23 lots at 9,991. But a rival bot had a 40-lot bid resting at 9,906, far below the touch, and 40 is more than 23. So the template took the rival's order for the wall, put its fair value at 9,957 against the recorded wall mid of 10,001.5, 44.5 below it, and spent the session selling into bids that were worth more than it thought. Its fair was 5 or more off on 36% of ticks (`IDEAS.md`, "The template against the teams' P4 traders in one market").

The fix was a different definition of "the wall": the first level from the top with at least 15 lots, which a large order far from the touch can't capture. With it, the same market made +34 thousand credits instead of −371 thousand, and on the products tested without rivals (Starfruit, Kelp, Osmium and Pepper Root) it stayed within 2.3% of the old rule (`IDEAS.md`, "Wall-mid definitions"). The template has changed since: it now smooths Osmium's fair and stops its losses, and the same market gives −3 thousand credits with the old rule and +70 thousand with the new one (`data/rival_fair.csv`). The lesson holds either way.

The idea

A market maker quotes around a *fair value*, its estimate of what the product is worth right now. Everything it earns is measured from that number, so the first question on any market is how to estimate it from the book.

The mid is noisy

The mid, halfway between the best bid and the best ask, is the obvious estimate, and on many books it is a poor one. The touch is often made of small orders that come and go: a 2-lot bid appears one tick above the rest, and the mid jumps half a tick without anything having changed. Figure 4 shows 300 ticks of Prosperity's Kelp.

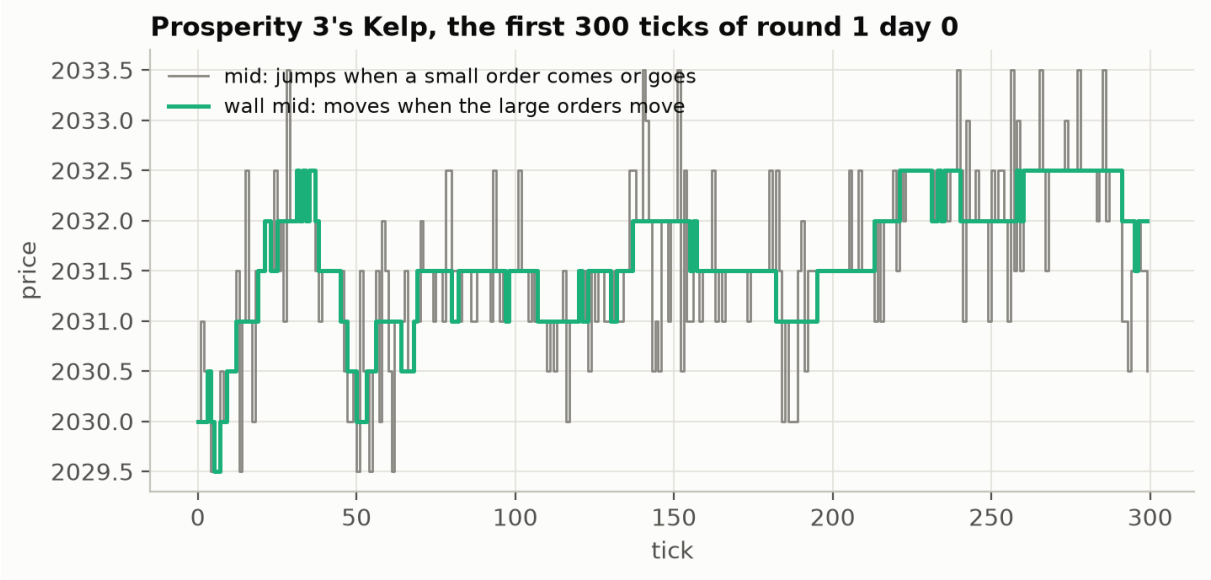


Figure 4: The mid and the wall mid over 300 ticks of Prosperity 3's Kelp. Data: `data/mid_and_wall.csv`, from the engine's loader.

The walls know more

On Prosperity, Frankfurt Hedgehogs found that each product's large orders, the walls, came from market-making bots that "appeared to know the true price and simply quoted rounded versions of it" (*their claim*, inferred from the website's PnL) (`references/prosperity3-frankfurt-hedgehogs/README.md:882-883`). Small orders from other bots sit inside them. So the midpoint of the walls tracks the value far better than the mid does:

$$\text{wall mid} = \frac{\text{bid wall} + \text{ask wall}}{2}$$

That is a fact about Prosperity's generator, not a law of markets. On ABIDES, an independent simulator whose traders have private views of a hidden value, every estimate we tried, mid, wall mid and the rest, was about 35 ticks from the value on average, and smoothing them made it worse (`IDEAS.md`, "An independent generator: ABIDES's RMSC04"). Which estimate works is something you measure on the day's data, not something you assume.

Robust beats clever

The story's lesson is about robustness. "The largest level" is an argmax, and one outlier captures it: anyone who can place a large order far from the touch moves your fair value. "The first level from the top with at least 15 lots", the template's `sized_mid`, ignores anything behind the real wall. It cost at most 2.3% when the book was clean, and saved everything when it wasn't. When other teams' bots share the book, or anyone has a reason to mislead you, prefer the estimate a single order can't move.

A fair value has a timescale

The other choice is how much history the estimate uses. A fair made from this tick's book alone follows every move; a smoothed one averages over many ticks and ignores short ones. Neither is

right in general. The right timescale is the one over which deviations from the fair *come back*.

Figure 5 shows how much that matters. On Prosperity 2's coconut coupon, an option (a contract whose value follows another product, here coconuts), the template valued the coupon at a model price plus the coupon's gap from that model, averaged with more weight on recent ticks (an exponential average over about `smile_window` ticks), and traded what was left of the gap. With a 20-tick window that fast part was large, about four times the half-spread, and slow to close (each tick kept 89% of the last, a half-life of about six ticks), and fading it lost 267 thousand credits. With a 5,000-tick window the same strategy made 442 thousand credits, its worst day +114 thousand (`IDEAS.md`, "Prosperity 2 round 4's coconut coupons with the template's options modes").

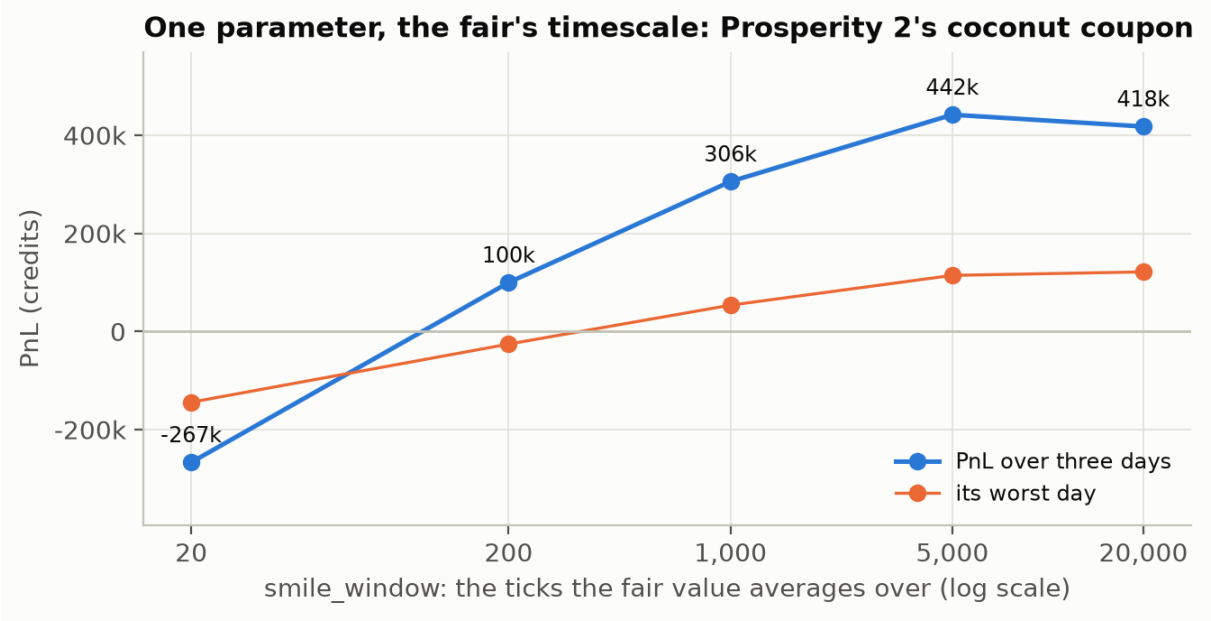


Figure 5: One parameter, the fair's timescale, on Prosperity 2's coconut coupon (take_edge 0.5). Data: `data/smile_window.csv`, from the IDEAS row's `arena.grid` command.

The check is cheap: before trading against a deviation, see whether it reverts at the horizon you trade it: its autocorrelation, or simply whether fading it made money on each day separately.

If you know ML

A fair value is an estimator of a latent variable from noisy observations, and the usual lessons hold: prefer a robust statistic when outliers can appear (a trimmed rule over an argmax), and choose the smoothing window to match the signal's timescale, as you would a filter's bandwidth. Where it breaks: the outliers here can be adversarial, placed by someone who profits from your error, and the latent "true value" may not exist at all on a market where informed traders, not a generator, set the price.

In the repo

- `fair_value` in `python/bots/template.py`, chosen per product by `PARAMS[...]["fair"]` (`mid`, `wall_mid`, `outer_mid`, `sized_mid`, `weighted_mid`, `hysteresis_mid`, `fixed`), with `wall_size` and `smoothing`.
- `arena.diagnose` says which estimate is least noisy on the day's data, and suggests a PARAMS entry with `--suggest`.

- The rows: “Wall-mid definitions”, “The template against the teams’ P4 traders in one market”.

! On the day

`RUNBOOK.md`, “Each round”, T+0:30: new products go into PARAMS by `references/wiki/concepts/reading-the-data-on-day-one.md`, which maps each `diagnose` note to a choice of fair value and timescale.

i **Go deeper**

- Stoikov, *The micro-price* (9 pp.): a fair value from the book’s imbalance, and why it beats the mid.
- Bouchaud et al., *Trades, Quotes and Prices*, ch. 7 §7.1–7.3 (PDF pp. 137–145): what the book’s volumes say about the next move.

Why does a market maker earn anything?

You'll need the book, the spread, taking and making, and fills (chapter 2), and the fair value (chapter 3).

The story

Put our template alone into Prosperity 3's round 1 and it makes 72,674 credits over three days. Put a second copy of the same bot into the same market, quoting the same prices, and each copy makes about 41 thousand: 41,352 and 41,848. With two copies beside it, each makes 27 to 30 thousand (Figure 6). Nothing about the market changed except that someone else was offering the same thing at the same price (`data/copies.csv`; the method in `IDEAS.md`, "The template against copies of itself").

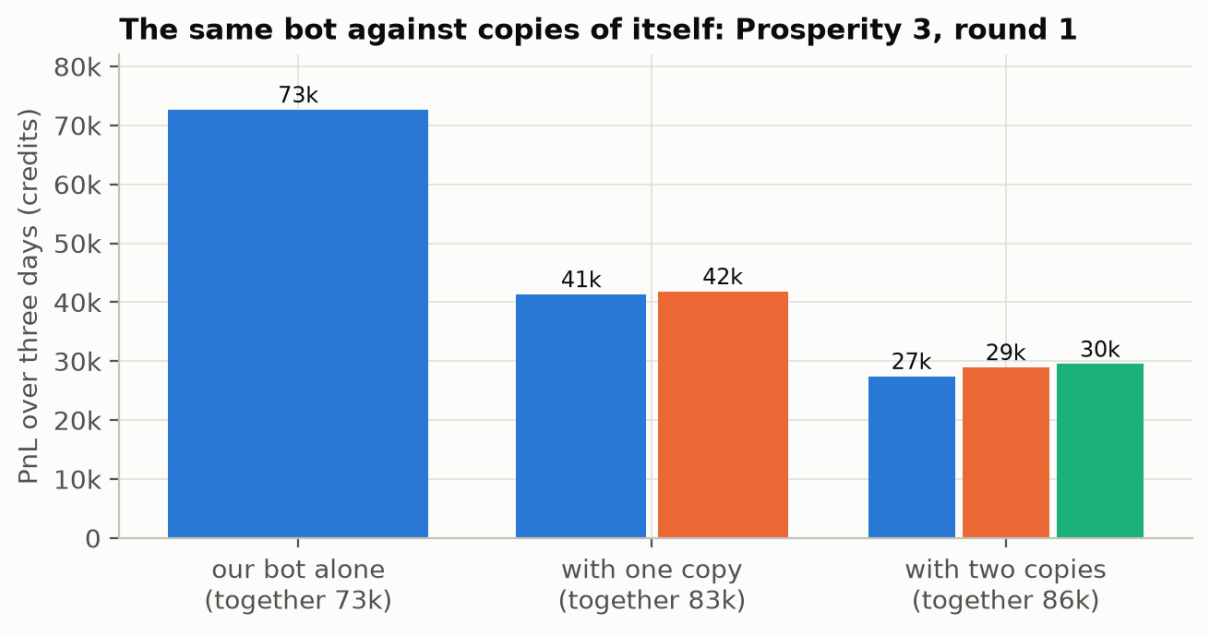


Figure 6: The same bot alone, against one copy of itself and against two, on Prosperity 3's round 1. Data: `data/copies.csv`, from `arena.backtest ... --vs python/bots/template.py`.

On Prosperity 4's Osmium the same measurement against other teams' bots gives the cleanest picture: the spread there is a pie of 55 to 62 thousand credits a round, and the bots split it. Our template kept 45 to 51% against one rival and 20% against three (`IDEAS.md`, "The template against the teams' P4 traders in one market"). A market maker's income is a rent, and rents are shared.

The idea

The spread is a fee for immediacy

Someone who wants to trade *now* crosses the book and pays the spread: they buy at the ask, above the mid, or sell at the bid, below it. Someone willing to wait posts an order and lets the hurried come to them. The market maker is the one who waits on both sides at once. Each time a taker buys from its ask and another sells to its bid, it has bought low and sold high by the width of its quotes, without needing any view of where the price is going.

Figure 7 shows this at small scale, the template's first 400 steps on one product of the Varde rehearsal. Most of its fills are passive, sells above the mid and buys below it, with some takes among them; its position swings between short 9 and long 6 and keeps coming back to zero; and its profit climbs in small steps, about €4 over 400 steps, while the price itself drifts down.

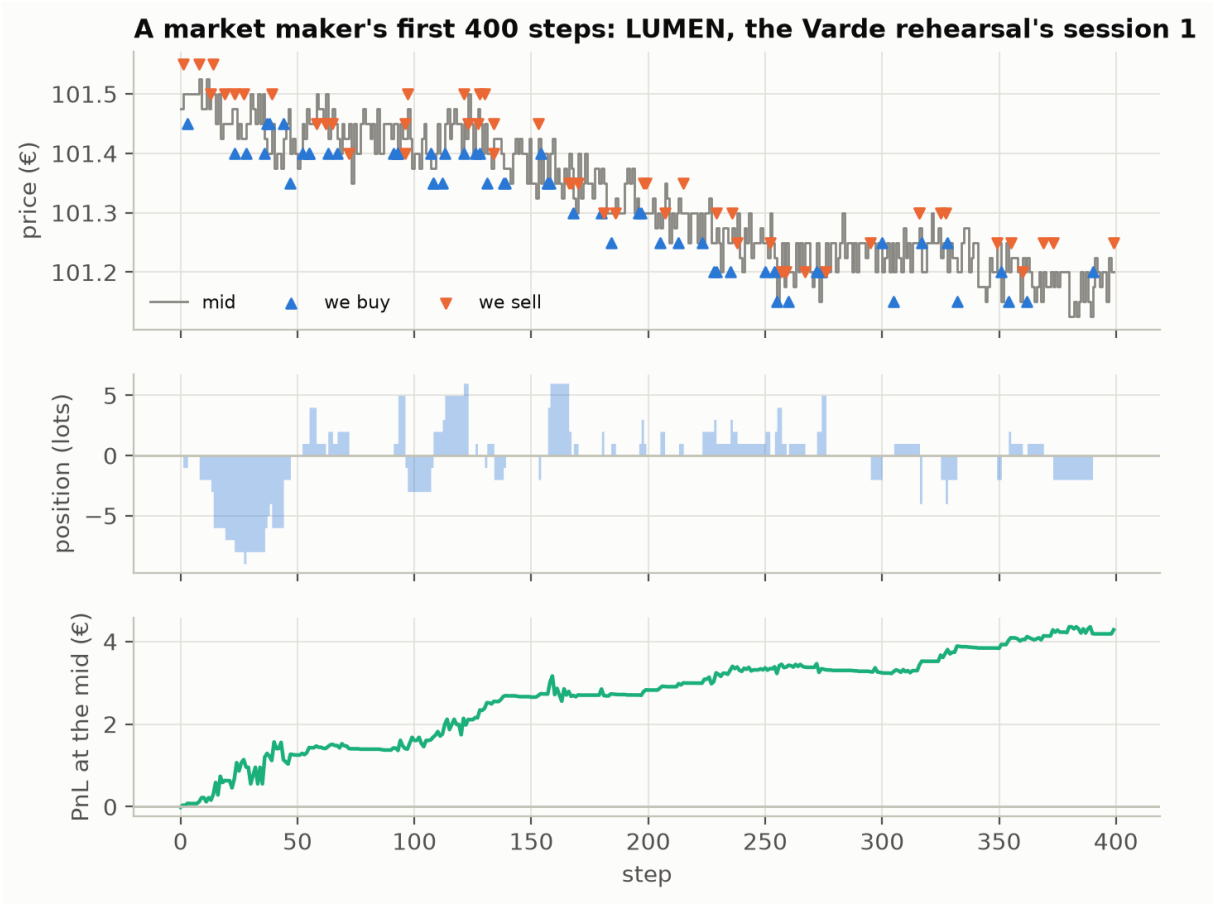


Figure 7: A market maker's first 400 steps: LUMEN, the Varde rehearsal's session 1. Top, the mid and our fills; middle, our position; bottom, PnL valued at the mid. Data: `data/round_trip.csv`, from the rehearsal portal's fills.

What can go wrong: the price moves after you trade

If that were the whole story, every market maker would be rich. The catch is what happens after a fill. When a taker buys from you, you are now short. If the taker knew something, the price rises, and the short loses more than the spread earned. So the question is not "how wide is the

spread" but "how wide is the spread compared with how far the price moves, on average, in the direction of the trades".

Trades, Quotes and Prices (eq. 17.13, PDF p. 345) gives the gain of a patient market maker, per unit traded:

$$\frac{\mathbb{E}[\mathcal{G}]}{\nu_0 T} = \frac{\mathbb{E}[s]}{2} - \mathcal{R}_\infty$$

In it, s is the spread and $\mathcal{R}_\infty$ is the long-run *impact*: after a buy, how far the mid ends up above where it was, on average, and the same for sells with the sign flipped (the book's eq. 1.9, PDF p. 35). Half the spread is what each fill earns at once; the impact is what the market takes back later. On the large stock markets the book studies, the two nearly cancel, and without the exchange's rebates the plain strategy loses (§17.2.2): making markets is a competitive business with thin margins.

Both sides can be measured from the data alone, before writing any bot, with the impact after 50 ticks standing in for the long run. On Prosperity 3's round 1 the margin is clearly positive: about 0.9 of a tick (a credit) a unit on Kelp and 1.4 on Resin, and trades barely move the price at all. On ABIDES it is negative, −0.32 of a tick (a cent) a unit, because its trades move the price by 1.3 ticks within 50 ticks (`IDEAS.md`, "Does market making pay here, by the textbook's test?"; "An independent generator: ABIDES's RMSC04"). That one measurement says whether plain market making can work on a market at all. Prosperity is the easy case, the textbook's "zero-intelligence" market where takers know nothing; chapter 5 is about the other kind.

So where does the income come from?

Three things decide it, and you control only some of them:

- **How much uninformed flow there is,** traders who cross the spread for reasons that don't predict the price. They are the market maker's customers. You can't create them.
- **How many others compete for them.** The story: a second bot at your price takes a little under half. Quoting a little better than the others wins the trades, but each step inside the spread gives away margin.
- **How well you avoid the informed.** Quoting further from the fair, pulling quotes when the book looks dangerous, or skewing them against your position (moving both quotes so the side that adds to it is the less attractive) all trade a little income for safety. Chapter 5.

One caution about Figure 6: between them the copies make a little more than one bot alone (83 and 86 thousand against 73). Don't read anything into that: a replay can't let the market react to two bots instead of one, so the total isn't something a real market would pay. Read each copy's share, which is what would be scored.

> **i** If you know ML
>
> Measuring $\mathbb{E}[s]/2 - \mathcal{R}_\infty$ from the data is estimating a baseline's expected reward before you train anything: if it is negative, no amount of tuning a plain market maker will fix it. It breaks in one place: it is measured on other traders' trades, and your quotes change who trades with you. A bot that quotes tighter than everyone gets more of the informed flow too.

In the repo

- The margin, per product: `uv run python -m arena.diagnose 1 --data p3` (its notes say "market making pays" or "thin or negative market-making margin", as a share of the spread).
- Copies and rivals in one market: `arena.backtest BOT DAYS --vs OTHER_BOT` (repeatable).
- The rows: "Does market making pay here, by the textbook's test?", "The template against copies of itself", "The template against the teams' P4 traders in one market".

On the day

`RUNBOOK.md`, "Each round", T+1:30: if the market is shared, backtest against copies and the round's rivals (`ON_THE_DAY.md`, "Is the market shared with the other teams?").

Go deeper

- Harris, *Trading and Exchanges*, ch. 14 "Bid/Ask Spreads", to the end of §14.4 (PDF p. 323): why spreads are as wide as they are.
- Bouchaud et al., *Trades, Quotes and Prices*, ch. 16 §16.1–16.2 and ch. 17 (PDF pp. 317–330, 338–350): the theory of the spread, and the profitability of market making.

Who is on the other side?

You'll need fills and passive orders (chapter 2), and half the spread against the impact (chapter 4). Fill models and grids are named here and explained in chapters 6 and 7.

The story

Everything in the template was tuned on Prosperity data. So we took it, unchanged, into ABIDES, an independent market simulator whose traders include a hundred "value" agents with noisy private views of a hidden price, and momentum traders who follow moves. ABIDES prices are in cents.

The template lost. Over ten days it made −21,798 cents (about −$218) under the most optimistic fill model, and −362,428 (about −$3,624) with orders that rest. Of a 36-cell grid of its settings, none made money with resting orders; doing nothing beat every one.

Then the test that explains it. We made twenty copies of the same ten days with each day's ticks put in a random order, same books, same trades, no sequence, and ran a small search over the bot's quoting distance on each. On the shuffled days the best setting made **+4,540,545 cents** on average, about +$45,000 (Figure 8). On Prosperity's Kelp the same test changes almost nothing: 27,390 credits on the real days, 27,326 on the shuffled ones (`IDEAS.md`, "An independent generator: ABIDES's RMSC04").

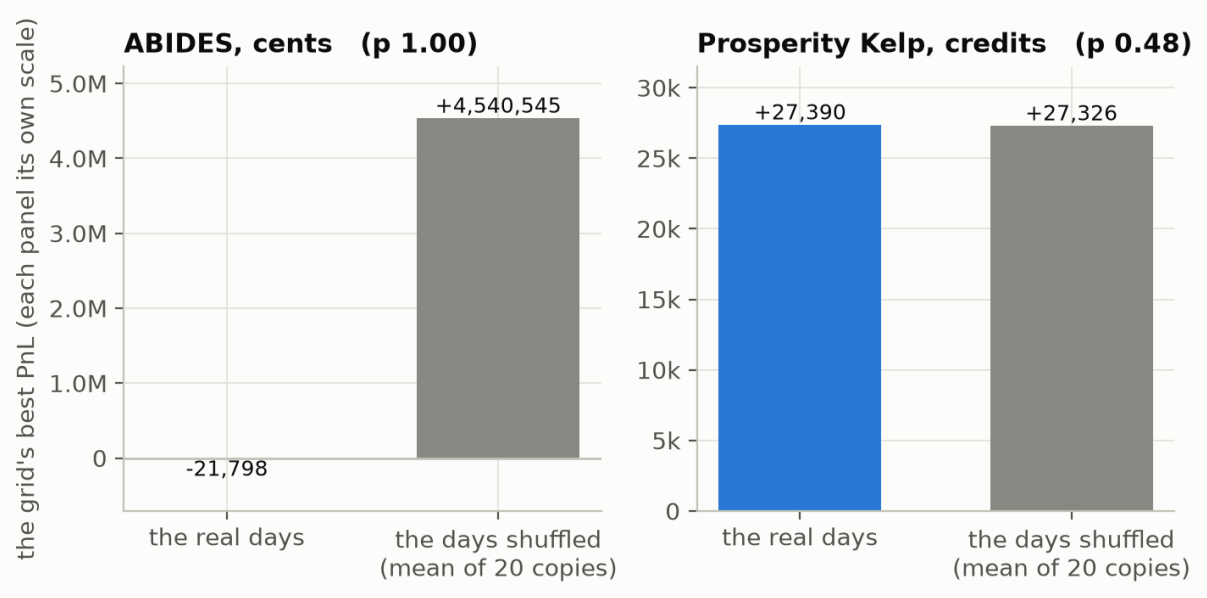


Figure 8: The grid's best PnL on the real days and on shuffled copies: ABIDES (cents, ten days) and Prosperity 3's Kelp (credits, three days); each panel on its own scale. Data: `data/shuffle.csv`, from `arena.grid ... --permute 20`.

Shuffling keeps every book and every trade and destroys the sequence: *what happens after* each fill, including moves that keep going. On ABIDES that was where the loss lived. The order of events was the adverse selection.

The idea

The fills you get are the ones you didn't want

A resting quote doesn't choose who trades with it. It is filled whenever someone finds it worth taking, and the people most eager to take it are the ones who know the price is about to move through it. A bid gets hit just before the price falls; an ask gets lifted just before it rises. This is *adverse selection*: conditional on being filled, a market maker's fills are worse than its quotes looked.

The measurement is the *markout*, a fill's profit some ticks later:

$$\text{markout}(h) = \text{side} \times (\text{mid}_{t+h} - \text{fill price})$$

with side $+1$ for a buy and -1 for a sell. Straight after the fill the markout of a passive fill is the edge it was quoted at, about half the spread. On an uninformed market it stays there as h grows. Under adverse selection it falls, and if it falls below zero the market maker pays to trade. On ABIDES our passive fills started at about $+2$ cents a unit; a single tick later the price had moved about 2 cents against them, leaving about nothing, and fills against the last move lost on every day (`IDEAS.md`, "An independent generator: ABIDES's RMSC04").

Two kinds of market

Prosperity's products are the friendly kind. On Kelp, where we measured it, the takers look close to random: our position's correlation with where the price went next was between -0.10 and $+0.18$ on Kelp, about nothing (`IDEAS.md`, "`skew` against Kelp's long position"). Kelp sits long much of the day for the same reason: more of its takers happen to sell, and the bot buys what they sell. That position is the flow, not a signal.

ABIDES is the other kind: some traders know something, and their trades move the price after them (chapter 4's impact, $+1.3$ ticks within 50). The template's whole playbook, quote close to a fair value and wait, was built for the first kind and loses on the second. The Arena may be either, or each product may be different, which is why the repo has a test for it before anything else is tuned.

When someone is informed, follow them

Adverse selection has a mirror image. If one trader is reliably informed, their trades are a signal. On Prosperity 3, one trader, named Olivia once round 5 showed names, bought at or near each day's low and sold at or near its high: in round 5 every one of her trades that the day left time to measure was followed by a move her way at 100, 1,000 and 5,000 ticks. Frankfurt Hedgehogs had spotted her from trade sizes before names were shown, and traded on her signal (`IDEAS.md`, "Following informed counterparties"; `references/prosperity3-frankfurt-hedgehogs/README.md:270-319`). Copying her on top of plain market making took our Croissants from about 2 thousand credits a day to 13 to 26 thousand.

The warning is chapter 7's story. In Prosperity 4, several teams each found a different "informed" trader, and none of those signals survived a day they hadn't been chosen on. A real Olivia is right on every trade; a lucky one is right for two days.

If you know ML

Adverse selection is selection bias with an adversary: the sample of fills you observe is chosen by the people who profit from your errors, so it is biased in the worst direction, and more so the more visible your quotes are. The shuffle test is a permutation test: destroy the temporal order and see whether the result needed it. Where it differs from the usual use: here the order costing you money is the finding, not a null to reject.

In the repo

- Markouts of any run: `uv run python -m arena.analysis results/` (from `arena.backtest ... --out results/`); by market state, `experiments.day.quote_markouts`.
- Prosperity-like or agent-like, per product: `experiments.day.premise` (the shuffle test and the impact, together).
- Informed counterparties: `experiments.day.counterparties`; the template's `copy` mode follows named traders.

On the day

`RUNBOOK.md`, "Each round", T+0:15 runs `experiments.day.premise` on the new days: whether each product is Prosperity-like or agent-like, and which grids to run (`ON_THE_DAY.md`, "What kind of market is each product?").

Go deeper

- Bouchaud et al., *Trades, Quotes and Prices*, ch. 11 §11.1–11.3 (PDF pp. 227–240): the response function, the impact after a trade.
- DeLise, *The negative drift of a limit order fill* (28 pp.): markouts, and why a backtest's fill rule decides them.

Why is my backtest lying?

You'll need passive orders and fills (chapter 2), the fair value (chapter 3), why making earns (chapter 4) and adverse selection (chapter 5).

The story

In Prosperity 4 a team that calls itself Kayzerjali tuned one product, Pepper Root, in a backtester. Over one window of 100,000 timestamps the backtest said +2,415; the live platform, on the same window, said +436 (*their claim*). They didn't conclude that the strategy was bad. They concluded that the backtester's fill model "diverges severely from the live platform", and rebuilt their loop around it: the live result became the only thing they optimised, and the backtester became a check for bugs before an upload (`references/online-snapshots/Kayzerjali_Prosperity2026/run_cycle.md:3-7`).

A Prosperity 3 team, Frankfurt Hedgehogs, said the same thing more politely. The open-source backtester they used, Jasper's, was "very accurate and highly flexible", but it "could not … accurately model potential fill probabilities after inserting liquidity for assets like Rainforest Resin and Kelp" (`references/prosperity3-frankfurt-hedgehogs/README.md:898-900`).

Both teams hit the same wall, and it is not a bug in anyone's code. It is a question that recorded data cannot answer.

The idea

A backtest replays a recorded day: every tick's order book and every trade that printed. Our bot reads each tick and sends orders, and the engine decides what would have happened to them. Some of that is easy. An order that crosses the book, buying at or above the best ask, trades with orders that were really there, at their prices: that is a *take*, and the replay knows the answer. The hard part is everything else.

A market maker's income comes mostly from orders that *don't* cross: a bid below the best ask and an ask above the best bid, waiting for someone to trade with them. These are *passive* orders. When the recording shows a trade at our bid's price, would it have traded with us? The recording can't say, because we weren't there. That trade happened between two other bots. If we had been there, the seller might have hit us instead, or might have hit the other bot first, or might not have traded at all because our order changed the book it saw.

So every backtest engine answers the question by assumption, and the assumption is a setting. Ours has four main ones, and Figure 9 shows what they do to one bot on the same three days.

- `all` (the default, which matches the open-source Prosperity backtester): a recorded trade at our price, or through it, fills us.
- `worse`: only a trade strictly through our price fills us. If someone sold below our bid, they would surely have sold to us first.

- `none`: no recorded trades at all; only orders in the visible book fill us.
- `resting`: orders stay in the book between ticks and wait in a queue behind the orders that were already at their price, so a trade at our price fills whoever was there first.

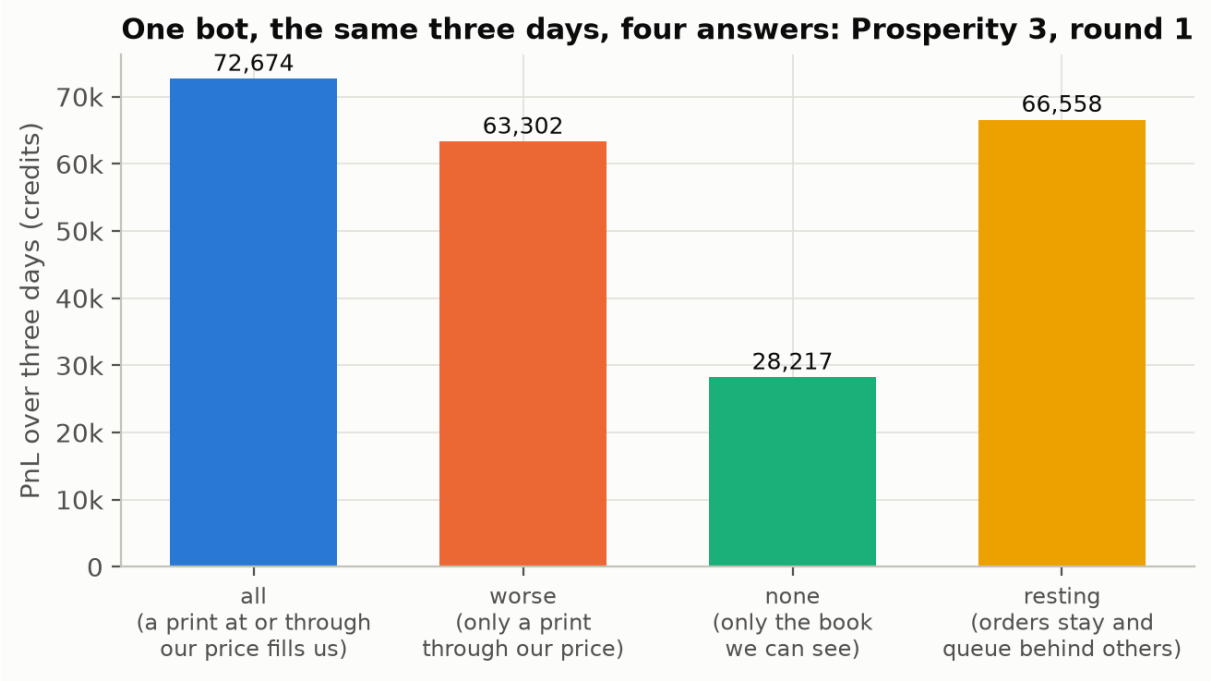


Figure 9: The template on Prosperity 3's round 1, all three days, under each fill model. Resting uses the default queue (`back`, behind everyone already there). Data: `data/fill_modes.csv`, from `arena.backtest python/bots/template.py 1 --data p3` in each mode.

The bot is the same, and so are the days. The answer goes from 28,217 credits to 72,674. More than half of the default number comes from fills that only exist if you believe that bots which traded with each other would have traded with us instead (`README.md`, "Fill assumptions matter").

Why the error isn't noise

If the fill model were only imprecise, it would cost some accuracy and nothing else. It is worse than that: it is biased, and the bias depends on the very parameters you tune.

Take Kelp's `make_edge`, how far from our fair value the bot quotes. The table gives what the template made with three settings of it on Prosperity 3's round 1, and on three days of round 5, in thousands of credits.

`make_edge`	0.5	1	1.5
under `all`	72.7	81.4	84.5
under `worse`	63.3	57.8	52.3
out of sample (round 5), `all`	53.8	61.1	64.5
out of sample (round 5), `worse`	48.8	44.3	38.8

Under `all`, the wider quote wins; under `worse`, it loses. And it isn't overfitting to three days: on round 5, which the choice never saw, each fill model ranks the edges the same way it did

in sample (`data/kelp_edges.csv`; `IDEAS.md`, "Why the zoo beat the template head to head"). Held-out data can't settle it, because the held-out days are replayed under the same assumption. **The fill model decides the edge.** A permutation test agrees: shuffling the order of Kelp's ticks finds the same best edge and the same lift over the median setting (a p-value of 0.55 for the best cell, 0.80 for its lift), so the choice comes from each tick's book and the fill rule, not from anything the price does over time (`IDEAS.md`, "A permutation test of the whole search").

A likely reason, which we haven't measured directly: a quote further from the fair sits where fewer traders reach, so a larger share of its fills are the doubtful kind, trades printed at its price between other bots. The rule that counts those as ours rewards moving out; the rule that doesn't punishes it.

The other things a replay assumes

The fill rule is the largest assumption, not the only one. Each of these is a setting in our engine, each was measured on Prosperity data, and none can be read from the data itself:

- **When an order arrives.** The replay matches an order against the book the bot saw. If the exchange applies it a tick later, the price has moved. The template on round 1 falls from 72,674 credits to 40,472 one tick late. Resin's takes against its fixed fair arrive after the price has already come back, and part of the loss is a race against the limit: a late order is checked again on arrival, against a position its own earlier orders have moved, and one failure rejects all the product's orders (on 3,248 of day 0's 10,000 ticks) (`IDEAS.md`, "Orders that arrive a tick late"; `--delay`).
- **Whether what we take stays gone.** A replay usually restores the recorded book every tick, so the volume we bought is back for sale a tick later. In our first rehearsal the bot made 42,446, in that round's units, on the hidden days with the book restored, and 31,172 when what it took stayed gone: a quarter of the score from one assumption (`IDEAS.md`, "The kickoff prompts on a disguised round"; `--deplete`).
- **Whether orders rest.** With orders that live one tick, plain market making on the jumpy basket legs of Prosperity 2 and 3 earned a few thousand credits each. Let the same quotes rest, and the next snapshot jumps straight through them, filling them at their own stale price: Prosperity 2's Roses lost 1,477 thousand (`IDEAS.md`, "Market making on products that jump, with resting orders").
- **What the position is worth at the end.** A bot that ends the day holding something is scored on cash plus that position at some price, and the engine chooses which:

  $$\text{PnL} = \text{cash} + \text{position} \times \text{mark}$$

  The mark can be the mid, the last trade or a portal's own average (the Varde rehearsal used the volume-weighted average of the last 100 trades, and didn't say so (`python/experiments/mock_round/round4/README.md`)). The `--mark` option picks the mid, the touch or the last trade; it has no such average.

What to do about it

Three habits follow, and the repo is built around them.

1. **Never trust one fill model.** Run every comparison under at least two, `all` and `worse`, and believe only what wins under both (`README.md`, "Fill assumptions matter"). If the ranking flips, the gain is a property of the assumption.

2. **Measure the real exchange early.** One upload of a plain bot tells you how far the portal is from each of your settings. That is chapter 8's tool, calibrate, and the playground before round 1 is the cheapest place to use it.
3. **Prefer the edges a replay can see.** A take against the visible book is filled the same way under every rule; income from passive fills is the part to doubt.

i If you know ML

A backtest is a simulator, and the fill model is its sim-to-real gap. Two things make it worse than the usual gap. It is *one-sided*: optimistic fills always look better, so a search drifts toward whatever the simulator flatters. And it is *correlated with the parameters*: `make_edge` decides how many of your fills are the doubtful kind, so tuning it is tuning to the gap. Held-out days don't help, as they would against overfitting, because they share the simulator. The fix is the one for any simulator: measure it against the real system and treat the remaining uncertainty as a range, here the span between `all` and `worse`.

In the repo

- The fill rules: `uv run python -m arena.backtest BOT DAYS --match-trades all|worse|none`, `--resting` with `--queue back|prob|front`, `--fill-ratio`, `--delay`, `--deplete`, `--mark mid|touch|last` (`python/arena/engine.py`; `ARCHITECTURE.md`, "Matching rules (Prosperity semantics)").
- Two models at once in a grid: `arena.grid ... --modes all,worse`; named bundles with `--scenario`.
- The rows: `IDEAS.md`'s "The template's market maker" (every fill mode), "Why the zoo beat the template head to head", "Orders that arrive a tick late".

! On the day

`RUNBOOK.md`, "Each round", T+1:00: while the fill mode is uncertain, the grid runs under the mode closest to the rules *and* `worse`; once the rules or the playground's fills pin it, under that mode, `worse` and `none`. `ON_THE_DAY.md`, "Which backtest settings match the playground?", says how to read the first upload against our settings.

i **Go deeper**

- Bouchaud et al., *Trades, Quotes and Prices*, ch. 21 §21.3–21.4 (PDF pp. 413–420): limit against market orders, the value of a place in the queue, and why backtests with phantom orders are biased.
- DeLise, *The negative drift of a limit order fill* (28 pp.): why the fills you get are, on average, the ones you wouldn't have wanted. That is chapter 5's subject, and the reason `worse` is closer to the truth than it looks.

How do I know a result is real?

You'll need fill models (chapter 6) and informed traders (chapter 5). From machine learning you'll need nothing new: this is the chapter where your background transfers most directly.

The story

Prosperity 4's round 4 showed teams the names of the bots behind each trade, "Mark 14", "Mark 22", "Mark 67" and so on, and every team went looking for an Olivia (chapter 5). Several found one. They didn't agree: one team's write-up names Mark 67, another Marks 14 and 55, another 14 and 38 (`references/wiki/contradictions.md`, row 3).

Ryan Challman's team fitted a weight for each named trader on the training days and used them to lean their fair value. On the day they had held out, the weights turned out to be tuned to particular days' behaviour; switching them off recovered about 15.8 thousand of held-out profit (*their claim*) (`references/prosperity4-ryan-challman/README.md:89-91`).

We ran the test that settles it: for each named trader, product and side, choose on two days, score on the third, three times. No Mark that traded more than a handful of times a day kept the sign of its move on all three days and beat half the spread. Names chosen on two days by their average move lost 31 thousand on the third. The careful teams were right to switch them off (`references/wiki/contradictions.md`, row 3).

Every one of those signals looked real on the days it was found on. That is the problem this chapter is about.

The idea

A grid is a hyperparameter search, with very few folds

A grid runs every combination of a few settings and ranks them. You know what happens next: the best of many noisy estimates is biased upward, the winner's curse, and the more combinations, the bigger the bias. What is unusual here is how little data there is. A Prosperity round gives three days; the Arena may give a handful. Three days are three folds, they are correlated, and the scored day may come from a different regime.

So the repo never ranks by one number. Every grid prints, for each cell:

- **`pnl`**, profit, the total over all days;
- **`min day`**, its worst day, because a setting that wins on average by winning big once is fragile;
- **`nbhd`**, the neighbourhood, the average over the cell and its neighbours, because a lone spike among poor neighbours is luck;
- **PnL under a second fill model**, usually `worse`, because chapter 6.

A setting near the top on all four is worth more than the single best `pnl`.

Hold out days, not ticks

The honest split is by day (Figure 10): choose on two days, score on the third, and repeat for each day. Splitting ticks at random would put a scored tick between two training ticks that share its price level and its regime, and every method would look good.

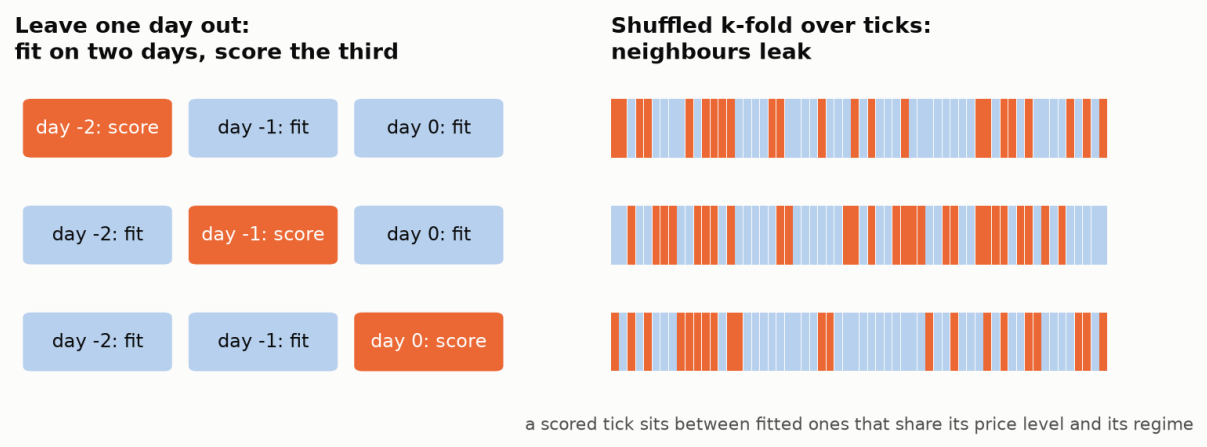


Figure 10: Leave one day out against shuffled k-fold. Days are the unit that can differ; ticks within a day share their regime.

Statistics for "the best of many"

Held-out days answer "does the choice survive a day it wasn't made on". Four more tests, each printed by `arena.grid`, answer "would the best of this many settings look this good by luck":

- **PBO, the probability of backtest overfitting** (Bailey et al., Definition 2.2, PDF p. 10). Cut the days into blocks, split the blocks into two halves in many ways (4,000 of the 48,620 possible halvings in the example below), pick the best cell on one half, and see where it ranks on the other:

  $$\text{PBO} = \Pr\left(\text{the in-sample best ranks below the median out of sample}\right)$$

  Near 0 is good; 50% means choosing was no better than picking at random.
- **DSR, the deflated Sharpe ratio** (a Sharpe ratio is the mean gain over its standard deviation): is the best cell's risk-adjusted gain larger than the best of that many cells with no gain would show?
- **SPA and StepM:** bootstrap tests of whether any cell, and then which cells, beat the bot as it is (its current settings), accounting for every cell tried. StepM's list at 4% is what the repo trusts.
- **The permutation test** (chapters 5 and 6): run the whole search on days with their ticks shuffled. If it finds as good a best, the choice doesn't need anything that happens over time.

One grid, read in full

Figure 11 is Kelp's two edges on Prosperity 3's round 1, under both fill models: `make_edge`, how far from the fair the bot quotes, and `take_edge`, how far the book must be mispriced before it takes.

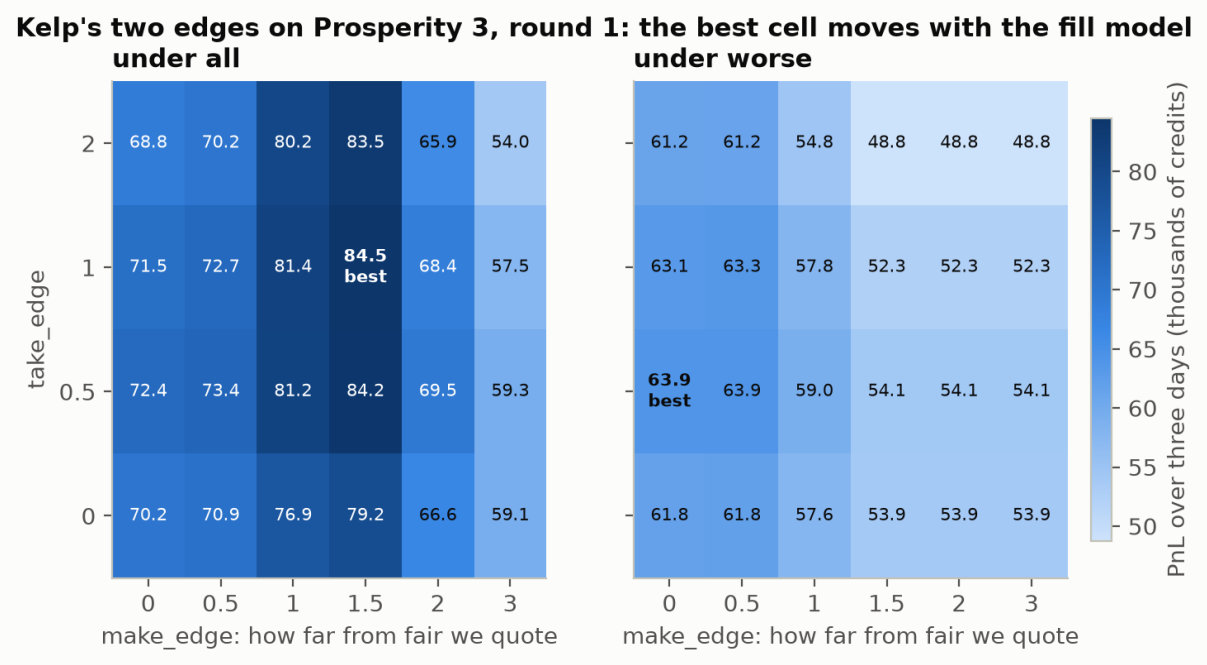


Figure 11: Kelp's `make_edge` against `take_edge` on Prosperity 3's round 1, PnL over three days (both products; only Kelp's settings change), under `all` and `worse`. Data: `data/kelp_grid.csv`, from `arena.grid ... --modes all,worse`.

Under `all` the case for quoting wider looks strong. The best cell, `make_edge` 1.5 with `take_edge` 1, is best by total, by worst day and by neighbourhood; it is chosen on every held-out day; PBO is 0%; SPA says some cell beats the bot as is (a p-value of 0.000) and StepM names nine. Only the DSR hesitates (0.74, under its 0.95 bar).

Then the second fill model. Under `worse`, only one of StepM's nine cells beats the bot as is, by 596. The repo's rule, the *pick*, takes StepM's best cell that beats the bot as is under every mode, and here that is a small step, `make_edge` 0.5 with `take_edge` 0.5: +736 under `all`, +596 under `worse`. Every statistic was right about the data it saw. None of them could see that the data was replayed under an assumption (the grid's own output, from `data/kelp_grid.csv`'s command).

When the profit is on one day

One more failure the tests don't catch: concentration. Ryan Challman's `ROBOT_DISHES` made about 389 thousand on one training day against 11 thousand and 4 thousand on the other two, 96% on one day; its mean reversion was strong on that day only. Rather than tune to that day, they halved its weight, a hedge against the scored day not looking like it (*their claim*) (`references/prosperity4-ryan-challman/README.md:115-141`). The `min day` column and the held-out days are there to show you such a profile before you size to it.

If you know ML

Everything here is model selection under multiple comparisons, and the tools are the ones you know: held-out folds, corrections for the number of trials, permutation nulls. Three things are harsher than usual. There are three folds, not a thousand. The folds are correlated and the test day may come from another distribution. And the simulator that scores every fold shares one bias (chapter 6), which no statistic computed inside it can detect, so the second fill model is part of the validation, not an extra.

In the repo

- `uv run python -m arena.grid BOT DAYS --grid PARAMS.P.key=v1,v2 --modes all,worse [--holdout last] [--permute 19]`: every measure above, the held-out days, PBO, DSR, SPA, StepM and the `pick` (`python/arena/grid.py`, `python/arena/stats.py`).
- The rows: "A permutation test of the whole search", "Selection rules on unseen days", "Following informed counterparties".

On the day

`RUNBOOK.md`, "Each round", T+1:00: the grid under two fill models, and a setting changes only to the summary's `pick`, with the one exception RUNBOOK names for round 1 if the rounds are scored on raw PnL.

Go deeper

- *Pseudo-mathematics and financial charlatanism* (Bailey et al., 34 pp.): why backtest overfitting is the default.
- The wiki's `references/wiki/concepts/validation-and-overfitting.md`, then *The probability of backtest overfitting* (34 pp.): CSCV and PBO in full.

What does the portal tell me?

You'll need PnL as cash plus position (chapter 2) and a backtest's assumptions (chapter 6).

The story

In the Varde rehearsal (chapter 1), the first upload scored €367.61 on the three public sessions. Our backtest of the same file, under its best guess of the exchange's rules, had said about €1,330, and calibrate, the tool that searches our engine's settings for one that reproduces the portal, refused: no setting came within 5% of every figure (Figure 12).

So the session read the portal's results file by hand. The rules had said there was a fee per order placed as well as per lot, and it was easy to underrate: the bot placed and cancelled orders all the time, and those fees came to €143 of a €511 gross, **28%** (`data/sessions.csv`; `python/experiments/mock_round/round4/README.md`). The fills file showed a second gap: the portal had filled far fewer lots than any of our settings that traded at all (8,466 against 11,345 to 18,772). The rule behind it, that only half of each trade reached resting orders, was not stated; it sat in the rehearsal's answer key. Neither was a strategy problem, and either would have misled every backtest that night (`python/experiments/mock_round/round4/results/DRYRUN_LOG.md`, "What calibrate printed").

The idea

The twelve figures

An upload comes back with a dozen numbers. For the Varde upload:

Figure	Varde upload	What it is
`score`	367.61	the PnL that ranks you: cash plus the positions at the mark, less any penalty
`fills`, `tradedVolume`	5,056; 8,466	how many fills, and how many lots
`inventory`	−6	the last session's end position, summed over products
`cash`	2,310.30	cash at the end, fees included
`markPrice`	233.44	the last session's marks, averaged over products
`breachPenalty`	0	the charge for going past a limit
`pnlSe`	22.98	the score's standard error

Figure	Varde upload	What it is
`absInventoryQ50` to `Q997`	8, 13, 27, 36	quantiles of the absolute position, pooled over the sessions

Three of them repay a closer look.

The score is an identity. It is cash plus position times mark, as in chapter 2, less any penalty, and you can check it. With one product and one session the check is exact. Here it fails twice over, and nothing is wrong: `cash` is summed over all three sessions while `inventory` and `markPrice` are the last session's, and `markPrice` is one number averaged over three products' prices. So 2,310.30 less 6 times 233.44 is not 367.61, and the identity holds only session by session and product by product. Know which figures are sums before you compare them.

`pnlSe` says how much the score is worth. On the Varde portal it is the standard error of the total over the sessions scored (`python/experiments/mock_round/round4/private/portal.py`):

$$\text{pnlSe} = \text{sd}(\text{session scores}) \times \sqrt{n}$$

The Varde sessions scored 110.21, 136.57 and 120.83: their spread is 13.3, times the square root of 3 is 23. Two uploads whose scores differ by less than a couple of standard errors are not shown to differ, however tempting the better number. With only a few uploads, this is what stops you chasing noise. What the Arena's `pnlSe` measures isn't known yet; calibrate tries per-tick, per-block and per-day versions and keeps the one that fits.

The position quantiles say how hard the bot leans on its limits. The quantiles sum the position over products, and the two products the bot traded allow 100 lots between them. A median of 8 lots and a 99.7th percentile of 36 mean the bot is usually near flat and at most uses about a third of its room; a bot whose median is near its limit is carrying a bet, whether it meant to or not.

Calibrate: fitting the simulator to the real thing

Our backtest has a dozen settings that recorded data can't decide (chapter 6): which trades fill us, whether orders rest and where they queue, delay, depletion, the mark, the share of each trade we can get. One upload of a plain bot measures them. Calibrate replays the uploaded file under every combination, computes the same twelve figures, and ranks the settings by their *worst* figure's error against the portal. If the best is within 5% on every figure, it prints the settings to paste into the day's format. If not, it refuses, and names the usual causes: the money off by one factor (a unit or scale mismatch), a cost the engine doesn't charge, or our wrapper doing something their exchange doesn't.

Figure 12 shows why it refused. The best-ranked setting barely trades, which makes it "least wrong" on cash and inventory and 98% wrong on the score. The setting closest on fills and volume, with half of each trade available, gets fills within 2.6%, but it scores €236.58 against the portal's €367.61 and ends the last session with a different position (−12 against −6), which throws cash off by over 300%. On top of that, the order fee is a cost the engine can't charge at all. No setting can be right everywhere when the exchange has rules the engine lacks, and a fit that is right on some figures and wrong on others would have been worse than none.

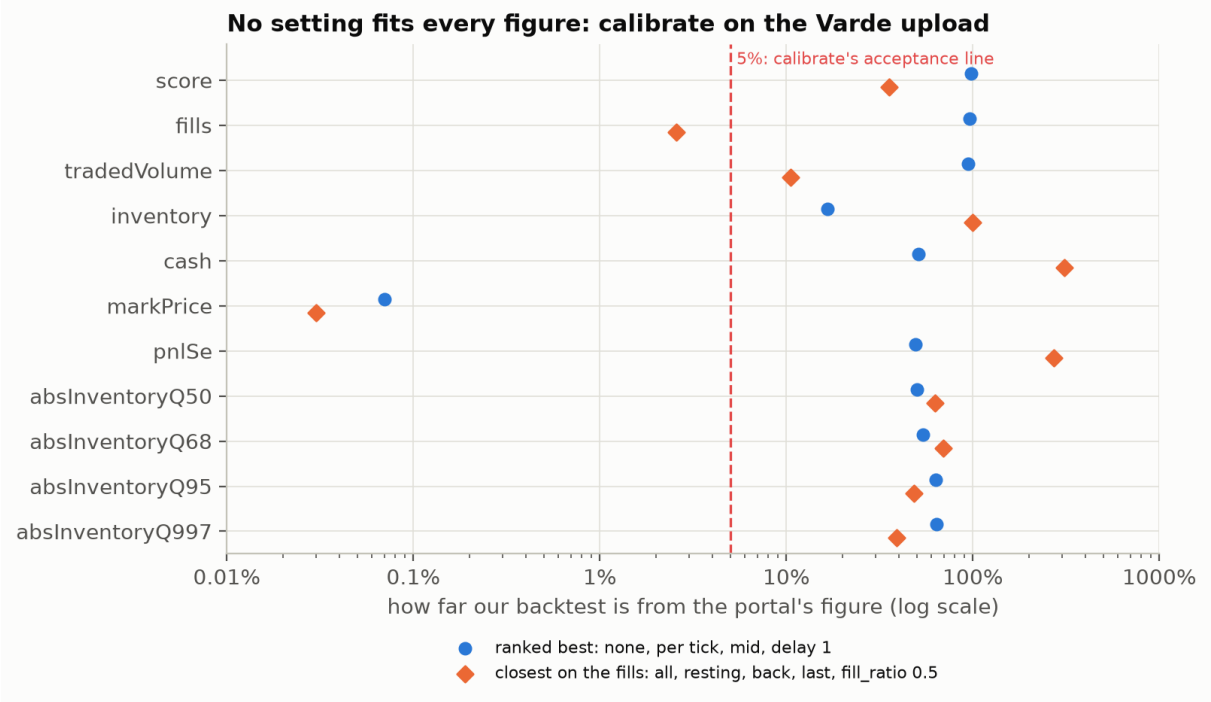


Figure 12: Calibrate on the Varde upload: each figure's error, for the best-ranked setting and for the setting closest on fills and volume alone, against calibrate's 5% line. Data: `data/calibrate.csv`, from `experiments.day.calibrate` through the round's plug-in.

If you know ML

Calibrate is system identification: fit a simulator's parameters so that its outputs match logs from the real system. Two things limit it. It sees twelve aggregate figures, not the hidden state, so very different settings can match the same totals, which is why it ranks by the worst figure and not the average. And the simulator's family may not contain the truth: refusing a bad fit is the same discipline as not deploying a model whose validation loss says it doesn't fit, however good its best metric looks.

In the repo

- `uv run python -m experiments.day.calibrate --bot BOT --days DAYS --format F --data D --portal metrics.json [--results results.zip] [--vary fill_ratio=0.25,0.5,0.75]`.
- Our backtest's figures in the portal's names: `arena.backtest ... --metrics`.
- The rehearsal: `python/experiments/mock_round/round4/` (README, and `results/DRYRUN_LOG.md`, "What calibrate printed").

On the day

`RUNBOOK.md`, "Each round", T+2:00: a validated bot at least as good as the last one, uploaded; its portal score against our backtest is the engine's error. `ON_THE_DAY.md`, "Which backtest settings match the playground?", says how to settle the engine's settings on that upload's figures and results file.

Go deeper

- The wiki's `references/wiki/concepts/submission-and-scoring-mechanics.md` and `probing-the-environment.md`: what other competitions' portals reported, and how teams measured an exchange they couldn't see.

How is the code laid out?

You'll need the round's loop (chapter 1), the order book and orders (chapter 2), fill models (chapter 6) and the portal's figures and penalty (chapter 8).

The story

The third rehearsal, the Kestrel Bourse, hid a rule: orders reached the exchange one step after the bot sent them. Two agent sessions played it, one with the repo before its calibration tools and one after. Both worked out the hidden rules soon after their first upload. Both lost money anyway: −4,850 and −283 credits on the public sessions, −3,553 and −489 on the hidden ones, where the exchange's own plain market maker made +325 (`IDEAS.md`, "The kickoff prompts on a disguised round").

Most of the loss wasn't the strategy. It was the few lines that turn a strategy's wishes into orders. With orders a step late, an order sent last step is neither in the book yet nor gone; neither session's code counted those orders "on their way". So each step it saw no order where it wanted one, sent another, ran into the exchange's cap of three working orders, and, when the late orders all arrived, held more than its limit and paid the penalty. Counting the orders on their way, and changing nothing else, took the public score from −283 to +142, the hidden one from −489 to −389, and the hidden days' penalty from 25 to 0. What was left was a choice of settings: one product was quoted at all, under a matching rule that made quoting it lose (`IDEAS.md`, "The kickoff prompts on a disguised round"). The sessions' bots lacked the counting in the code between the strategy and the exchange; the template now does it in EXECUTION (`on_the_way` and `went`), switched on by the ADAPTER's `DELAY`.

That is why the bot is split the way it is.

The idea

The engine knows no format

Figure 13 is the repo on one page.

- **The engine** (`python/arena/engine.py`) replays a `Day` against a bot and returns fills and PnL. It knows nothing about any exchange: no file layout, no bot interface, no units.
- **A format plug-in** (`python/arena/formats/<name>.py`) knows one exchange. It reads that exchange's files into Days, wraps the exchange's bot interface so the engine can call it, and states the engine settings the exchange's rules fix (resting orders, the mark, delay, fees). On the day the Arena gets its own plug-in, `qminers.py`, already sketched with `TODO(day)` markers.
- **The tools** (`arena.backtest`, `arena.grid`, `arena.stress`, `arena.validate`, `arena.diagnose`, `experiments.day.calibrate` and the day's experiments) all take `--format` and `--data`, and work on any exchange that has a plug-in.

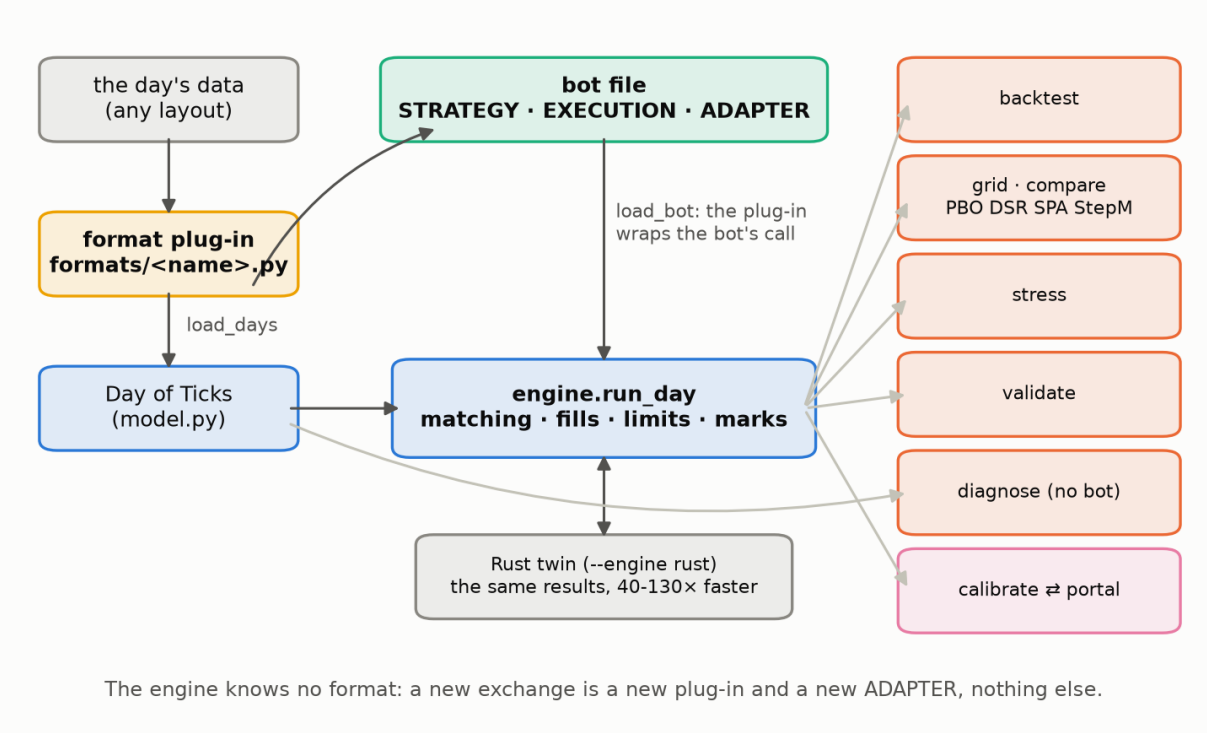


Figure 13: The repo's architecture. A format plug-in turns any exchange's data into Days of Ticks, and wraps the bot's call; the engine and every tool work only on those. Drawn from `docs/guidebook/research/repo_map.md`, A.

So a new exchange costs a plug-in and an adapter, not a new set of tools. Chapter 1's rehearsal wrote both in about five minutes.

The bot in three parts

The template, `python/bots/template.py`, is one self-contained file, as the competitions require, in three sections:

- **STRATEGY** decides what the bot *wants*: for each product, a fair value (chapter 3), then *take* (cross the book where it is mispriced against the fair), *clear* (trade back towards zero at the fair when the position is large), and *make* (quote either side of the fair). Other modes trade towards a target instead: a level the price returns to, a basket's spread, an option's model price, a trader worth following. Every choice is a parameter in `PARAMS`, one entry per product, over shared `DEFAULTS`.
- **EXECUTION** turns what the strategy wants into orders and cancels, given what already rests, what is on its way, and the limits. This is the part the Kestrel story was about. It is generic, the same on every exchange.
- **ADAPTER** is the only code that knows the exchange: it reads the exchange's state into the template's own form, and writes its orders back in the exchange's format. On the day it is the part that changes.

Keeping the three apart is what lets one strategy run on every exchange we have rehearsed, and what keeps an execution bug from hiding inside strategy code, where the Kestrel sessions looked for it first.

The rest of the repo

- **The Rust twins** (`rust/`): the engine, the simulator, the statistics and the bots ported to Rust, giving the same results to the bit, about 130 times faster per backtest and 40 to 80 times for a whole grid (`rust/README.md`). `--engine rust` uses them; a twin whose Python changed since it was checked is refused.
- `IDEAS.md`: every idea tested on Prosperity data, with the command and the numbers, kept even when it failed ("dropped here"): priors to re-test on the Arena's data, not verdicts.
- **The wiki** (`references/wiki/`): concept pages that join the literature, the other teams' write-ups and our measurements, each line cited.
- **The day's documents**: `RUNBOOK.md` (one page), `ON_THE_DAY.md` (every open question, with what to assume and what changes if it's different) and `KICKOFF_PROMPT.md` (what starts an agent session).

> **If you know ML**
>
> The split is the one between a model, its inference wrapper and a data loader. The engine is the evaluation harness, plug-ins are dataset loaders behind one interface, STRATEGY is the model, EXECUTION is the serving logic, and ADAPTER is the input and output layer for one deployment target. As with serving code, the last two are the hardest to test offline: a replay with no delay can't show an order-counting bug that only a delay exposes. Where the analogy breaks: a model's serving layer rarely changes the model's results, while here EXECUTION decides which orders exist at all, so it is part of the strategy's measured performance.

> **In the repo**
>
> - `ARCHITECTURE.md`: how the parts fit, and its paragraph "Formats are plug-ins" before writing one.
> - The template's sections: search `template.py` for `STRATEGY`, `EXECUTION` and `ADAPTER`; `on_the_way` and `went` count orders on their way.
> - `arena.validate` replays the bot one step late by default (`--late STEPS` changes it), to catch the Kestrel bug before an upload.

> **On the day**
>
> `ON_THE_DAY.md`, "The format swap, step by step": the plug-in, the adapter and the checks, in order, as the one checklist for the swap.

> **Go deeper**
>
> - `ARCHITECTURE.md` in full, then `python/arena/engine.py`'s module docstring: the matching rules in each mode.

What do I do on the day?

You'll need the whole guide; this chapter only explains why the day runs as `RUNBOOK.md` says. Every instruction is RUNBOOK's, and if this chapter and RUNBOOK ever disagree, RUNBOOK is right. Written 4 November 2026, from what the organisers had said by then.

The story

Qminers' own 2024 hackathon, a sports-betting contest, ended with an in-person final of three rounds, scores reset, a shorter horizon. Before it, the eventual winner wrote a small simulation of the final: twenty finalists with models of varying quality, correlated with each other, each with its own appetite for risk. He used it to choose how much of his bankroll to bet, choosing the fraction that won most often, not the one that made most on average.

He took the lead in the first round and extended it in the second. Before the third he noticed that its data came from just after covid, a period other finalists' models might handle better than his, and halved his stake so as not to lose the lead. It turned out to be "a minor mistake": his model "performed the best during this time", and at full stake he estimates he would have ended near 143 thousand points. He kept "a dominant first place lead" anyway (`references/qminers-hackathon-2024/README.md:41-63`).

The point is not that halving was right. It is that the question changes once you are ahead: from "what makes the most" to "what keeps the place".

The idea

The weekend

Saturday starts with registration at 08:00, as a whole team. The rules are published at the opening, and the *playground*, 10:30 to 12:00, is the one chance to try the real exchange before anything is scored. Then four rounds of five hours, at 13:00, 19:00, 01:00 and 07:00, which the site weights 10, 20, 30 and 40% (unconfirmed). There is no elimination; every team plays all four. AI tools are allowed; outside help is not (`ON_THE_DAY.md`, "Known so far" and "The day: timeline and roles").

Why the round runs as it does

RUNBOOK's "Each round" is the loop from chapter 1, with times on it (Figure 14). Each step has a reason in this guide:

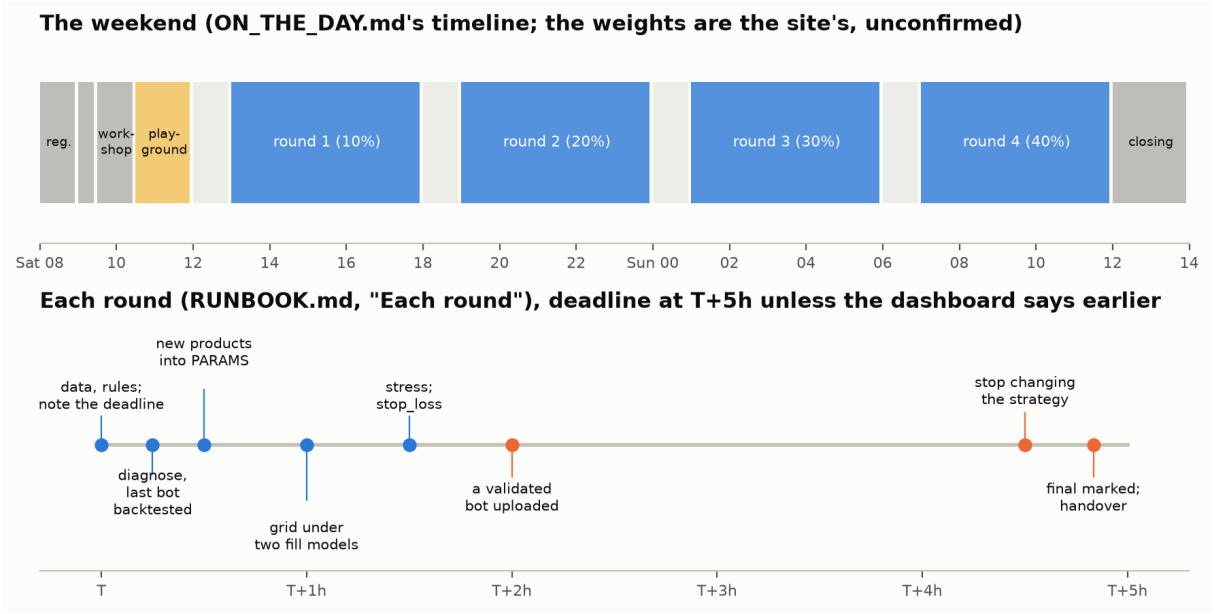


Figure 14: The weekend and one round. The weekend from `ON_THE_DAY.md`'s timeline, the round from `RUNBOOK.md`'s "Each round"; both drawn from those files by `make_data.py`.

- **Data and rules first, then `diagnose` and the premise test** (T+0:00 to T+0:15): before tuning anything, find out what kind of market each product is (chapters 3 and 5). For an agent-like product, a bot that does nothing is the benchmark, and the first candidates are other kinds of fair value and a gate on moves (`ON_THE_DAY.md`, "What kind of market is each product?").
- **The grid under two fill models** (T+1:00): until the playground or the rules settle which fill model the Arena has, every choice must hold under both (chapters 6 and 7); after that, under that model, `worse` and `none`. A setting changes only to the grid's `pick`, with one exception RUNBOOK names for round 1.
- **A validated bot at least as good as the last one, uploaded, by T+2:00**: its portal score against our backtest is the engine's error (chapter 8). In rounds 2 to 4 the round's first upload is last round's bot, the baseline to beat, from which the backtest's settings are fitted again (`ON_THE_DAY.md`, "Every round").
- **Stop changing the strategy half an hour before the deadline**, or earlier if twice the first evaluation's time is longer: the last change is the one with the least evidence behind it, and an upload that fails can't be marked final.
- **The handover note ten minutes before the deadline**: what was learned, for the next round's session, which starts fresh.

Above every step sit RUNBOOK's "Rules that hold all day": never upload code `arena.validate` hasn't passed, and never a `GRAY_AREA` bot without an organiser's yes; a round's first upload is a bot that has already run in the playground or last round's final bot; never choose between two variants by a portal score that differs by less than the engine's error; one evaluation at a time, and an upload that completes becomes final, so mark the chosen one final again after any exploratory upload.

Each round, choose again

The site promises "a forgiving market" first and "a sophisticated finale". So the round-1 bot is a starting point for round 2, not a default: RUNBOOK has each round re-run the premise

test, re-grid every setting on the round's own data, and try one alternative to the last round's approach. Every number in this guide was measured on someone else's market. Each is a prior for the day, to be re-tested on the Arena's data, round by round.

Risk, once there is a score to protect

Two things decide how much risk to carry into later rounds, and neither is known yet: how the four rounds combine into the final ranking, and whether profit carries over. The day's timeline has the decision made in writing at breakfast, before round 4, from the standings, the weights and the lead (`ON_THE_DAY.md`, "The day: timeline and roles"). The story above is the frame for that decision: once you are ahead, the variance you could avoid counts for more than it did, and when you are behind, for less.

> **If you know ML**
>
> A competition ranked on one final evaluation rewards the probability of being first, not the expected score, the way a leaderboard with a private test set rewards robustness over the public score. Where the analogy breaks: on a leaderboard you choose the model once, while here the stake can change from round to round, after you have seen the standings, so the choice of risk is itself a decision made on new information.

> **In the repo**
>
> - `RUNBOOK.md` (one page), `ON_THE_DAY.md` (every open question about the Arena) and `KICKOFF_PROMPT.md` (the prompts that start each round's session).
> - `experiments.day.standings` and `experiments.day.leaderboard_curves`: what the standings say about risk, once there are standings.

> **On the day**
>
> `RUNBOOK.md`, start to finish. This chapter is only the reasons.

> **Go deeper**
>
> - The wiki's `references/wiki/concepts/risk-and-protecting-a-lead.md` and `harder-rounds.md`: how other teams sized risk with a lead, and how competitions make later rounds harder.

Glossary

Each term with the chapter that defines it, and its nearest neighbour in machine learning where there is an honest one.

Adapter The part of the bot that knows one exchange: it reads the exchange's state and writes its orders. Chapter 9. *ML: the input and output layer for one deployment target.*

Adverse selection The tendency of a waiting order to be filled just before the price moves against it, because the traders keenest to take it know something. Chapter 5. *ML: selection bias, chosen by an adversary.*

Ask, bid An offer to sell, an offer to buy. The best ask and best bid together are the *touch*. Chapter 2.

Backtest A replay of recorded days with the bot trading in them. Chapter 6. *ML: offline evaluation in a simulator.*

Bot as is The bot with its current settings, the baseline a grid's cells are tested against. Chapter 7.

Calibrate The tool that searches the engine's settings for one that reproduces the portal's figures, or refuses. Chapter 8. *ML: system identification.*

Clear Trading back towards the target position at the fair value; the target is flat unless a strategy sets one. Chapter 9.

Delay Orders reaching the exchange some steps after the bot sends them; the engine's `--delay`. Chapters 6 and 9.

DSR, SPA, StepM Tests of whether the best of many settings beats the bot as is by more than luck would give: the deflated Sharpe ratio, Hansen's test of superior predictive ability, and Romano and Wolf's stepwise test. Chapter 7. *ML: corrections for multiple comparisons.*

Engine The code that replays recorded days against a bot and decides its fills: `python/arena/engine.py`. Chapters 6 and 9.

Fair value The bot's estimate of what a product is worth now; every quote is placed relative to it. Chapter 3. *ML: an estimate of a latent variable.*

Fill A trade that executes one of your orders, wholly or partly. Chapter 2.

Fill model The engine's assumption about which recorded trades would have filled orders that weren't really there: `all`, `worse`, `none`, or resting with a queue. Chapter 6. *ML: the simulator's sim-to-real gap, one-sided.*

Format plug-in The module that reads one exchange's data into Days of Ticks and wraps its bot interface. Chapter 9. *ML: a dataset loader behind a common interface.*

Held-out day A day left out when a setting is chosen, then used to score it. Chapter 7. *ML: a validation fold.*

GRAY_AREA A flag in a bot file marking a strategy that may break the spirit of the rules; `arena.validate` fails such a file, so it is never uploaded without an organiser's yes (RUNBOOK, "Rules that hold all day"). Chapter 10.

Grid Every combination of a few settings, each backtested and ranked. Chapter 7. *ML: grid search over hyperparameters.*

Impact How far the price moves, on average, in the direction of a trade after it; $\mathcal{R}_\infty$ in the long run. Chapter 4.

Level All the orders at one price on one side of the book. Chapter 2.

Limit order An order to buy at a price or lower, or sell at a price or higher. Chapter 2.

Making, taking Making is posting an order that waits; taking is crossing the book to trade at once and paying the spread. Chapter 2.

Mark The price at which a position is valued when PnL is computed. Chapters 2 and

6.

make_edge, take_edge How far from the fair value the bot quotes, and how far the book must be mispriced before it takes. Chapters 6 and 7.

Market maker A trader that quotes on both sides and earns the spread from those who want to trade now. Chapter 4.

Markout A fill's profit some ticks later, measured from the mid. Chapter 5.

Mid Halfway between the best bid and the best ask. Chapter 2.

Order book The orders waiting at a moment, bids on one side and asks on the other, grouped by price. Chapter 2. *ML: the state.*

PBO The probability of backtest overfitting: how often the best setting on one half of the data ranks below the median on the other. Chapter 7. *ML: a measure of model-selection overfitting.*

Pick The grid's chosen setting: the best cell that StepM, a multiple-testing correction, confirms beats the bot as is under the grid's first fill model, and that also beats it under every other model the grid ran. Chapter 7.

PnL Profit and loss: cash plus the position valued at the mark. Chapter 2. *ML: the reward, after a choice of reference price.*

Position, limit What you hold, positive if long, negative if short; the most the exchange lets you hold either way. Chapter 2.

Premise test The repo's check of whether a product is Prosperity-like (uninformed takers), agent-like (informed traders, moves that continue) or mixed. Chapter 5.

Queue The order in which waiting orders at one price are filled: earliest first. Chapter 2.

Resting An order that stays in the book between ticks until filled or cancelled. Chapter 2.

Settlement price The price a portal values what is still held at the end, the Varde Exchange's name for its mark. Chapter 2.

Shuffle test The search re-run on days with their ticks in random order; if the result doesn't drop, it didn't need the order of events. Chapters 5 and 6. *ML: a permutation test.*

Spread The gap between the best ask and the best bid. Chapter 2.

STRATEGY, EXECUTION The template's two generic parts: what the bot wants (fair value, take, clear, make), and the orders and cancels that get it there. Chapter 9.

Template The repo's bot, `python/bots/template.py`: one file, with settings per product. Chapter 9.

Tick One step of the recorded market; also the smallest price step, the *tick size*. Chapter 2. *ML: a time step.*

Wall, wall mid The largest level on each side of the book, in the engine's definition; the wall mid is their midpoint. On Prosperity the walls are a market-making bot's large, stable quotes, a fact about its generator. Chapters 2 and 3.